

MLOps Assignment 01: End-to-End Production ML Pipeline

Course: Machine Learning Operations AIMLCZG523

Project: Heart Disease Classification and Serving API

Dataset: UCI Cleveland Heart Disease dataset

GitHub username: Rajvik85

Recommended repository URL: <https://github.com/Rajvik85/MLOPS-Assignment-1>

Objective: Build a reproducible, tested, tracked, containerized, deployable, and monitored ML API for heart disease prediction.

This report summarizes the implemented MLOps solution and maps the repository evidence to the assignment rubric.

Rubric area	Marks	Repository evidence
Data acquisition and EDA	5	src/download_data.py, src/eda.py, notebooks/01_eda.ipynb, plots/eda_*.png
Feature engineering and models	8	src/preprocess.py, src/train.py, sklearn Pipeline, GridSearchCV
Experiment tracking	5	MLflow logging, mlruns, metrics, plots, model artifacts
Packaging and reproducibility	7	models/best_model.joblib, requirements.txt, Dockerfile
CI/CD and tests	8	tests/, pytest.ini, .github/workflows/mlops_ci.yaml
Containerization	5	FastAPI app, Dockerfile, .dockerignore, sample JSON
Deployment	7	kubernetes/deployment.yaml, kubernetes/service.yaml
Monitoring/logging	3	API logs and /metrics endpoint
Documentation/report	2	README.md, Report.md, Study_Notes.md, this PDF

1. System Architecture

The solution follows a practical MLOps flow: source code and data preparation feed model training; model runs are tracked in MLflow; the selected model is saved as a reusable sklearn Pipeline; FastAPI serves predictions; Docker packages the service; Kubernetes deploys the container; logs and Prometheus metrics support monitoring.

- Developer commits code to GitHub.
- GitHub Actions installs dependencies, lints, tests, validates training, and validates Docker build.
- Data is downloaded from UCI and cleaned through reusable preprocessing code.
- Training compares Logistic Regression and Random Forest with GridSearchCV.
- MLflow logs parameters, metrics, ROC curves, confusion matrices, and model artifacts.
- FastAPI loads models/best_model.joblib and exposes /health, /predict, and /metrics.
- Kubernetes runs two API replicas with readiness and liveness probes.

Architecture flow:

GitHub -> CI/CD -> Data ingestion -> Preprocessing Pipeline -> Model Training -> MLflow Tracking -> Saved Model -> FastAPI -> Docker -> Kubernetes Service -> Client Requests and Metrics

Key production-readiness decisions:

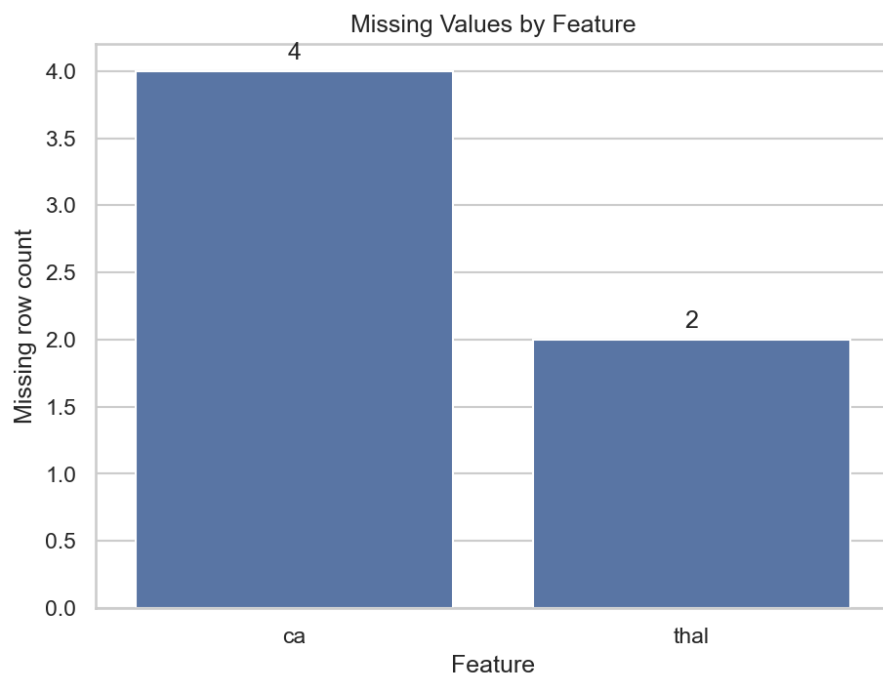
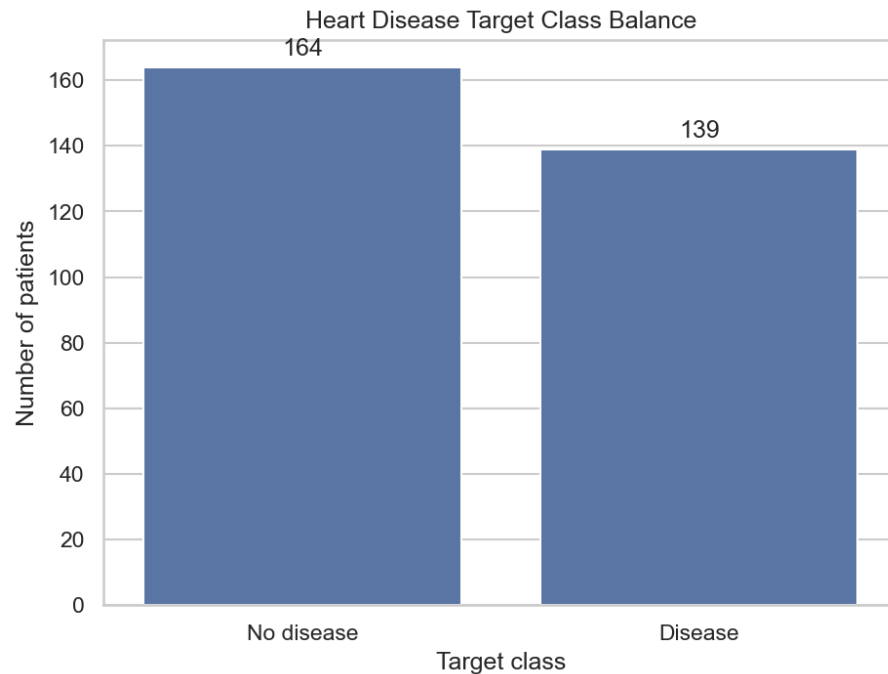
- A single sklearn Pipeline prevents training-serving skew.
- The API validates every request with Pydantic before inference.
- The Docker image runs as a non-root user.
- Kubernetes probes automatically detect unhealthy pods.
- The /metrics endpoint supports Prometheus scraping.

2. Data Acquisition and EDA

The dataset is fetched from the UCI Machine Learning Repository by `src/download_data.py` and stored as `data/raw/processed.cleveland.data`. The raw file contains 303 patient rows and 14 columns. The original target values 1 to 4 are mapped to binary class 1, while target value 0 remains class 0.

Important EDA findings:

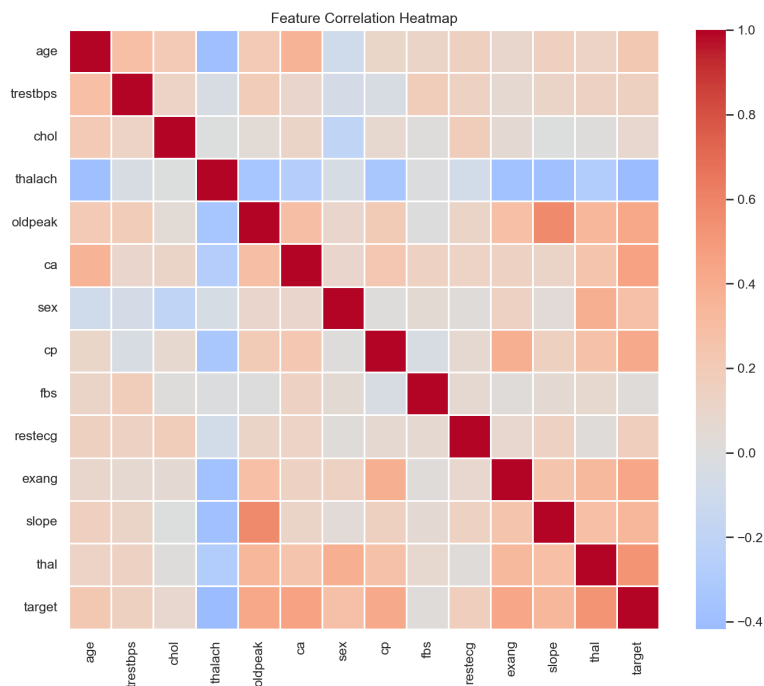
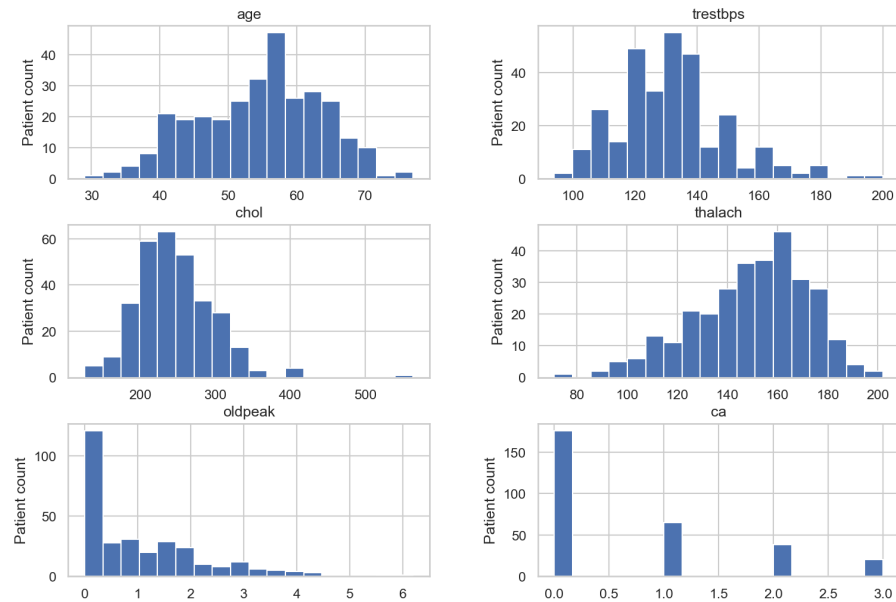
- The target is reasonably balanced: 164 no-disease rows and 139 disease rows.
- Missing values are encoded as question marks and mainly appear in `ca` and `thal`.
- Several clinical features, especially maximum heart rate, `oldpeak`, `ca`, `thal`, and chest pain type, show useful relationships with the target.



3. EDA Visualizations

The FAQ asks for professional visualizations including histograms, correlation heatmaps, class distribution plots, missing value analysis, and feature relationship analysis. The script `src/eda.py` regenerates all required plots from the raw dataset.

Numerical Feature Distributions



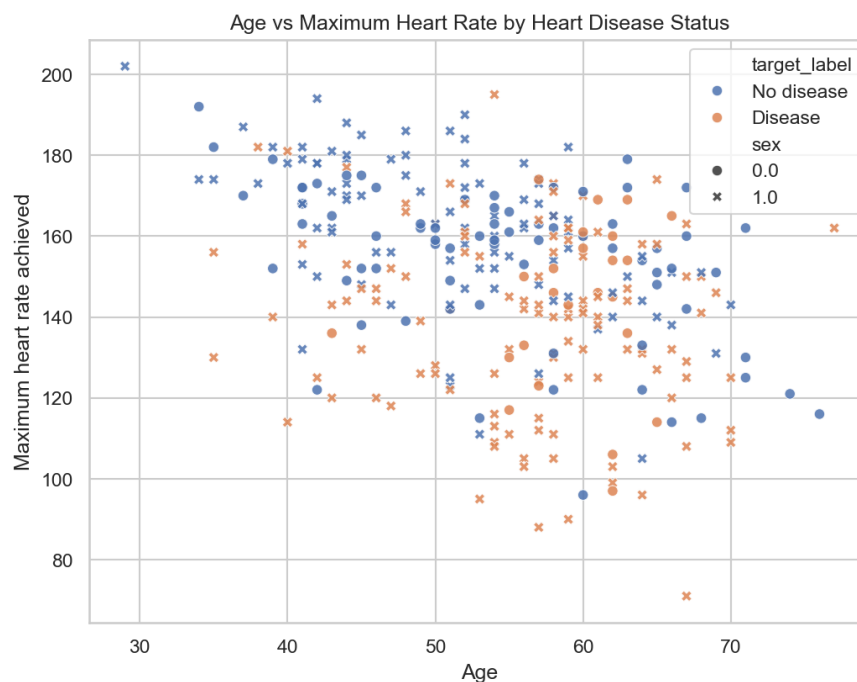
4. Feature Engineering and Preprocessing

The preprocessing layer uses a ColumnTransformer so numerical and categorical columns receive appropriate transformations. This transformer is embedded in the saved sklearn Pipeline, which is essential for reproducible inference.

Feature group	Columns	Processing
Numerical	age, trestbps, chol, thalach, oldpeak, ca	Median imputation and StandardScaler
Categorical	sex, cp, fbs, restecg, exang, slope, thal	Most-frequent imputation and OneHotEncoder(handle_unknown=ignore)
Target	target	Map 0 to 0 and values 1 to 4 to 1

Why this matters:

- Median imputation is robust for clinical numerical features with outliers.
- Most-frequent imputation is simple and appropriate for categorical medical codes.
- StandardScaler supports Logistic Regression by normalizing numerical feature scale.
- OneHotEncoder avoids imposing fake ordinal distance on categorical codes.
- handle_unknown=ignore makes inference more resilient to unseen category values.



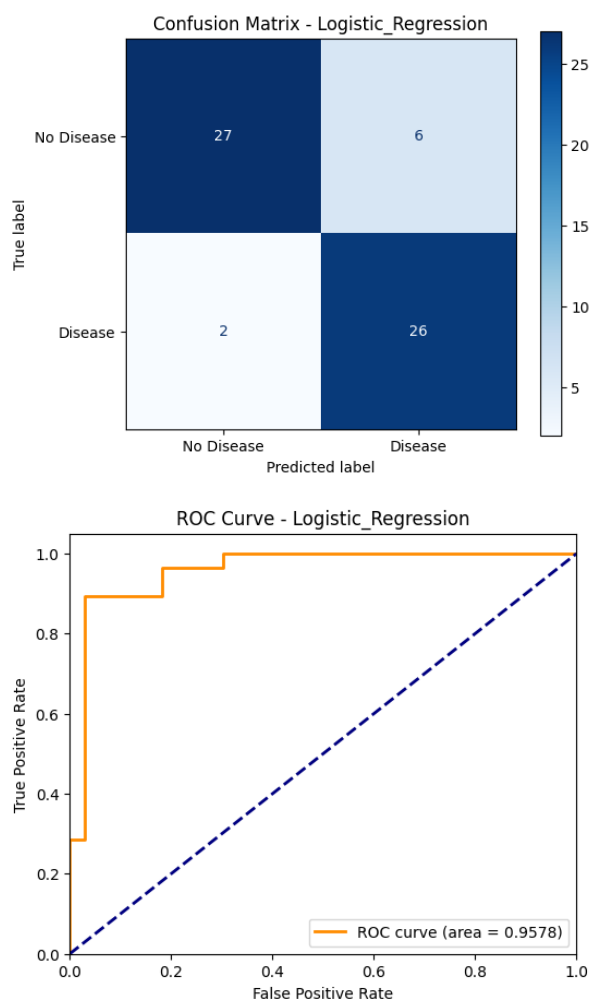
5. Model Development and Selection

The project trains at least two required classifiers: Logistic Regression and Random Forest. Both are tuned with GridSearchCV using 5-fold cross-validation and ROC-AUC scoring. The command `python -m src.train --model both` trains both models and saves the best by ROC-AUC.

Model	Best hyperparameters	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	C=1.0, solver=lbfgs	0.8689	0.8125	0.9286	0.8667	0.9578
Random Forest	depth=5, split=10, trees=100	0.8689	0.8333	0.8929	0.8621	0.9426

Model selection:

Logistic Regression is selected as the production model because it has the highest ROC-AUC and F1-score while matching Random Forest on accuracy. Its interpretability is also useful in a healthcare-risk context.



6. Experiment Tracking and Packaging

MLflow tracks all training runs under the Heart_Disease_Classification experiment. Each run logs the selected hyperparameters, evaluation metrics, confusion matrix, ROC curve, and model artifact. This makes the model selection process auditable and reproducible.

The local production artifact is models/best_model.joblib. It stores the full sklearn Pipeline, including both preprocessing and classifier. This means the API can accept raw patient JSON and apply the same transformations used during training.

Reproducibility controls:

- requirements.txt pins direct dependencies to compatible major versions.
- The random_state is fixed for deterministic train/test split and model behavior.
- The model file is saved in a standard reusable joblib format.
- The README provides clean setup commands from a new virtual environment.
- The .gitignore prevents local virtual environments and cache files from polluting GitHub.

MLflow screenshot requirement:

Before final submission, capture screenshots showing the experiment list, model metrics, ROC curves, confusion matrices, and logged model artifacts.

7. API, Testing, and CI/CD

The serving layer is implemented with FastAPI. It exposes GET /health, POST /predict, and GET /metrics. Pydantic validates the request schema, and Prometheus instrumentation exposes metrics for monitoring.

Sample local API commands:

```
uvicorn src.api:app --host 127.0.0.1 --port 8000
curl http://127.0.0.1:8000/health
curl -X POST http://127.0.0.1:8000/predict
-H "Content-Type: application/json"
--data @examples/patient_request.json
```

Automated testing:

- tests/test_preprocess.py validates data cleaning and preprocessing pipeline output.
- tests/test_api.py validates health, prediction, and schema validation behavior.
- tests/test_eda.py validates EDA plot generation.
- tests/test_train.py validates plot generation and best-model selection logic.

CI/CD workflow:

- black checks formatting.
- flake8 checks lint and syntax quality.
- pytest runs unit and integration-style tests.
- The training pipeline is validated on CI.
- Docker image build is validated on CI.

8. Containerization and Kubernetes Deployment

The Dockerfile builds a production API image using python:3.10-slim. The image copies dependencies, source code, and the trained model. It runs as a non-root appuser and exposes port 8000.

Docker proof commands:

```
docker build -t heart-disease-api:latest .  
docker run --rm -p 8000:8000 heart-disease-api:latest
```

Kubernetes deployment:

- deployment.yaml creates two API replicas for availability.
- RollingUpdate prevents downtime during replacement.
- Readiness and liveness probes call /health.
- Resource requests and limits protect cluster capacity.
- service.yaml exposes the API through a LoadBalancer service.

Kubernetes proof commands:

```
kubectl apply -f kubernetes/deployment.yaml  
kubectl apply -f kubernetes/service.yaml  
kubectl get pods && kubectl get svc heart-disease-api-service
```

For local testing, use kubectl port-forward service/heart-disease-api-service 8000:80, then call the same /predict endpoint.

9. Monitoring, Logging, and Final Evidence

Monitoring is implemented through API request logs and Prometheus-compatible metrics. Logs show model loading, prediction requests, prediction output, and shutdown events. The /metrics endpoint exposes counters and runtime metrics that Prometheus can scrape.

Monitoring proof commands:

```
curl http://127.0.0.1:8000/metrics
kubectl logs deployment/heart-disease-api
```

Final evidence checklist:

- Use <https://github.com/Rajvik85/MLOPS-Assignment-1> as the final repository URL if you create the repo with the recommended name.
- Add screenshots for MLflow, GitHub Actions, Docker, Kubernetes, /predict, and /metrics under screenshots/.
- Record a short video showing repository, tests, MLflow, API prediction, Docker/Kubernetes deployment, and monitoring.
- Do not upload venv, __pycache__, .pytest_cache, or local editor files.

Conclusion:

The project implements the required MLOps pipeline end to end: data ingestion, EDA, preprocessing, tuned model training, experiment tracking, reusable model packaging, API serving, automated tests, CI/CD, Docker containerization, Kubernetes deployment manifests, and monitoring hooks. The remaining final-submission work is to attach runtime screenshots and video evidence from the user's GitHub, Docker, and Kubernetes environment.